

Unit – 2

Requirement Engineering & Design Concept:

Understanding the Requirements, Requirement Modeling, Requirement Specification, and Requirements Engineering: Seven distinct tasks of requirement engineering, Gathering functional/non-functional requirements, Create SRS.

Software Design and Architecture: Design concepts and design principle, Architectural design, design UML diagrams: (Use case, class, sequence, activity diagram)

Requirement Engineering & Design Concept

Understanding the Requirements

In software development, **requirements** describe what a system should do and the constraints under which it must operate. Requirements represent the **needs and expectations of users, customers, and stakeholders**.

Understanding requirements is the **first and most important step in software engineering**. If requirements are misunderstood, the final system may not satisfy the user's needs.

Requirements are typically divided into two major types.

Functional Requirements

Functional requirements describe **the specific functions or services the system must perform**. They define the behaviour of the system.

Examples:

- The system shall allow users to login.
- The system shall allow customers to book tickets.
- The system shall generate reports.

Non-Functional Requirements

Non-functional requirements describe **quality attributes or constraints of the system**. They specify **how the system performs its functions**.

Examples:

- Performance (system response time)
- Security
- Reliability
- Usability
- Scalability

For example:

"The system shall respond within 2 seconds."

Requirement Engineering

Requirement Engineering (RE) is a systematic process used to **identify, analyse, document, validate, and manage the requirements of a software system throughout its lifecycle**.

Requirement engineering ensures that the **software developed meets the expectations of stakeholders and users**. The requirement engineering process includes **seven distinct tasks**.

Seven Distinct Tasks of Requirement Engineering

1. Inception
2. Requirement Elicitation
3. Elaboration
4. Negotiation

- 5. Requirement Specification
- 6. Requirement Validation
- 7. Requirement Management

1. Inception

Inception is the **initial stage of requirement engineering** where the project begins.

In this stage:

- Stakeholders are identified.
- Business objectives are defined.
- System scope is determined.

Example:

In an online examination system, stakeholders include students, administrators, and teachers.

2. Requirement Elicitation

Requirement elicitation is the process of **gathering requirements from stakeholders**.

Common elicitation techniques include:

- Interviews
- Questionnaires
- Workshops
- Brainstorming
- Observation
- Prototyping

The result of this stage is a **collection of functional and non-functional requirements**.

3. Elaboration

Elaboration involves **analysing and refining gathered requirements**.

During elaboration:

- Requirements are organized and structured.
- System models such as UML diagrams are created.
- Inconsistencies and missing requirements are identified.

4. Negotiation

Sometimes different stakeholders may have **conflicting requirements**. Negotiation helps resolve these conflicts.

The main objectives are:

- Prioritizing requirements
- Balancing cost, time, and resources
- Reaching agreement among stakeholders

5. Requirement Specification

Requirement specification is the process of **documenting requirements formally**.

The result of this process is a document called:

Software Requirement Specification (SRS)

The SRS document serves as a **reference for system design, development, and testing**.

6. Requirement Validation

Requirement validation ensures that the requirements are:

- Correct
- Complete
- Consistent
- Feasible

Validation techniques include:

- Reviews
- Prototyping
- Test case generation
- Stakeholder approval

7. Requirement Management

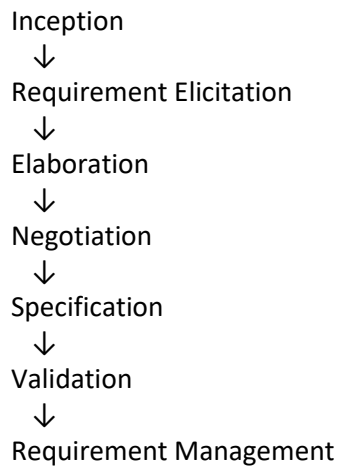
Requirement management deals with **handling requirement changes during the project lifecycle**.

Activities include:

- Change control
- Version management
- Requirement traceability
- Impact analysis

Requirement management helps prevent **scope creep** and keeps the project under control.

Requirement Engineering Flow



Requirement Modelling

Requirement modelling is the process of **representing system requirements using diagrams and models** so that stakeholders and developers can understand the system clearly.

Requirement models help:

- Visualize system functionality
- Detect missing requirements
- Improve communication

Different types of requirement models include:

Scenario-Based Modelling

Scenario-based modelling describes **how users interact with the system in real situations**.

Example:

Use Case Diagram

Data Modelling

Data modelling describes **data entities and relationships within the system**.

Example:

Entity Relationship Diagram (ERD)

Flow-Oriented Modelling

Flow-oriented modelling shows **how data flows through the system**.

Example:

Data Flow Diagram (DFD)

Class-Based Modelling

Class-based modelling represents the **object-oriented structure of the system**.

Example:

Class Diagram

Behavioural Modelling

Behavioural modelling shows **how the system behaves over time or in response to events**.

Examples:

- Sequence Diagram
 - State Diagram
 - Activity Diagram
-

Requirement Specification

Requirement specification is the process of **documenting all system requirements in a structured and clear manner**.

The output document is called the **Software Requirement Specification (SRS)**.

SRS acts as an **agreement between the client and the development team**.

Software Requirement Specification (SRS)

The **SRS document** provides a detailed description of the software system.

Structure of an SRS Document

1. Introduction

Includes:

- Purpose of the system
- Scope of the system
- Definitions and terminology

2. Overall Description

Provides general information about the system including:

- System overview
- User characteristics
- Assumptions and dependencies

3. Functional Requirements

These describe **what the system should do**.

Example:

- The system shall allow users to register.
- The system shall allow users to login.
- The system shall generate reports.

4. Non-Functional Requirements

These describe **quality attributes of the system**.

Examples include:

- Performance
- Security
- Reliability
- Availability

5. External Interface Requirements

Defines interactions with other systems.

Types include:

- User Interface
- Hardware Interface
- Software Interface

Importance of SRS

The SRS document is important because it:

- Serves as a contract between client and developer
- Acts as the basis for system design and development
- Helps in project planning and estimation

- Reduces development errors
-

Software Design and Architecture

Software Design

Software design is the process of **transforming requirements into a blueprint for building the software system**.

Software design focuses on:

- System architecture
 - Data structures
 - Interfaces
 - Components
-

Software Design Concepts

Software design concepts are fundamental ideas used to structure software effectively.

Abstraction

Abstraction means **showing only essential features while hiding unnecessary implementation details**.

Example:

ATM machine hides internal banking operations.

Modularity

Modularity means **dividing the system into smaller independent modules**.

Benefits:

- Easier maintenance
- Easier debugging
- Improved reusability

Cohesion

Cohesion refers to **the degree to which elements within a module belong together**.

High cohesion is desirable because it means the module performs a **single well-defined task**.

Coupling

Coupling refers to **the degree of dependency between modules**.

Low coupling is desirable because it allows modules to operate independently.

Software Design Principles

Design principles guide developers in creating **high-quality and maintainable software systems**.

Important principles include:

DRY (Don't Repeat Yourself)

Avoid duplication of code.

KISS (Keep It Simple)

Software design should be simple and easy to understand.

High Cohesion

Modules should perform one specific function.

Low Coupling

Modules should have minimal dependency on each other.

Software Architecture

Software architecture defines the **overall structure of the system and the relationships between its components**.

Architecture determines:

- System organization
- Component interactions
- Data flow

Types of Architecture

Layered Architecture

The system is divided into layers such as:

- Presentation Layer
- Business Logic Layer
- Data Layer

Client–Server Architecture

In this architecture:

- The client sends requests
- The server processes requests and returns responses

Example:

Web applications.

MVC Architecture

MVC stands for:

- Model (data)
- View (user interface)
- Controller (logic)

MVC separates the application into different components for better maintainability.

UML Diagrams

UML (Unified Modelling Language) is a graphical language used to **visualize, specify, and document software systems**.

UML diagrams help developers understand the structure and behaviour of a system.

1. Use Case Diagram

A Use Case Diagram shows **the interaction between users (actors) and the system**.

Actors represent external entities interacting with the system.

Example:

Actors:

- User
- Admin

Use Cases:

- Login
- Book Ticket
- Make Payment

2. Class Diagram

A Class Diagram represents the **static structure of a system**.

It shows:

- Classes
- Attributes
- Methods
- Relationships between classes

Example:

Class: User

Attributes: userID, name

Methods: login()

3. Sequence Diagram

A Sequence Diagram shows **the interaction between objects over time**.
It represents how messages are exchanged between system components.
Example flow:
User → Application → Server → Database → Response

4. Activity Diagram

An Activity Diagram represents the **workflow or process flow of a system**.
It shows:

- Activities
- Decisions
- Flow of control

Example workflow:
Start → Login → Select Item → Make Payment → End

Conclusion

Requirement engineering and software design are **critical phases of the software development lifecycle**.
Requirement engineering ensures that the system requirements are correctly understood and documented, while software design transforms these requirements into a structured blueprint. Techniques such as requirement modelling, SRS documentation, architectural design, and UML diagrams help developers build reliable, scalable, and maintainable software systems.